

Module 4: Lesson 1

Recurrent Neural Networks (RNNs)



Outline

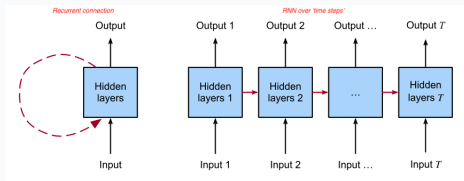
- ▶ What are RNNs and why are they useful?
- ▶ Working with sequences
- ▶ Latent auto-regressive models
- ▶ Recurrent Neural Networks

What are RNNs and why are they useful?

We already know how to build neural networks that work with data of different lengths and characteristics such as images, but we haven't dealt with sequential data yet:

- What should we do when the order of the input data matters for the prediction?
 - Sequence of images → Video!
 - Sequence of words → Text! → Natural Language Processing (NLP)

Recurrent Neural Networks (RNNs) are deep learning models that capture the dynamics of sequences via recurrent connections—i.e., cycles in the network.



Working with sequences

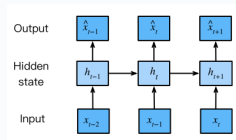
The fact that a specific sequence of input data matters for the prediction of the next is very intuitive:

- what is behind the auto-complete feature of your phones when you text?

However, working with sequences in a deep neural network has some challenges. In order to properly understand what RNNs do, let's start from a baseline: **autoregressive (AR) models**.

- ▶ AR models aim to predict a future value x_t given the past values of this variable: $\mathbb{E}[x_t | x_{t-1}, \dots, x_{t-n}]$
- ▶ Problem #1: How many past observations do we need? → in Latent AR models h_t summarizes the past

$$\hat{x}_t = \mathbb{P}(x_t | h_t), \text{ where } h_t = g(h_{t-1}, x_{t-1})$$

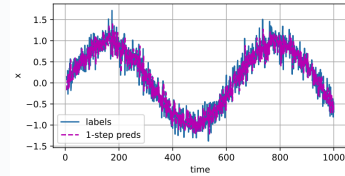
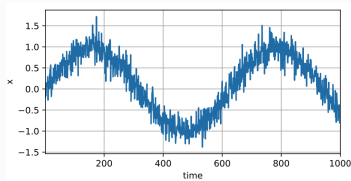


- ▶ Problem #2: Markov condition? → Given recent history, future conditionally independent of the past?

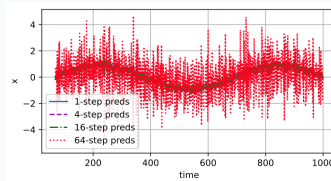
$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t | x_{t-1}, \dots, x_1)$$

Working with sequences

Suppose we want to predict the next logical step of a sin function with some noise like the one on the left. Then, we can use linear regression to predict the one-step-ahead values:



How about predicting 4 steps ahead? and 16? 64?



RNNs for NLP

One of the main uses of RNNs nowadays is in natural language processing (NLP). Unfortunately, the scope for applying NLP into quantitative finance problems is, for now, very limited.

We will not see in depth the use of RNNs for NLP during this course, but please note that the functioning of the whole network model that we will learn will be equivalent to that for NLP.

There is just one thing within the NLP framework that will need to be taken care of: **tokenization**:

- ▶ Tokens are the atomic (indivisible) units of text.
- ▶ A token can be entire words, characters, or even sentences.
- ▶ Still, tokens are simply strings that need to be integrated within a **vocabulary** in numerical form
- ▶ Once we have transformed text into sequence data we can simply train language models as estimates of the joint probability of a text sequence!

You can learn more about this in sections 9.2 and 9.3 in chapter 9 of Zhang et al.'s book *Dive into Deep Learning*.

Recurrent Neural Networks (RNNs)

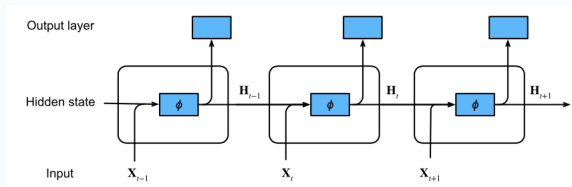
RNNs are neural networks **with hidden states**, similar to the latent AR model.

- First, let's revisit briefly how an NN **without hidden states** (e.g., MLP) worked:

$$H = \phi(XW_{xh} + b_h) \rightarrow O = HW_{hq} + b_q$$

- In an NN **with hidden states**, we save the hidden layer output (H_{t-1}) with a weight (W_{hh}) to account for the sequence. So, for time step t :

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h) \rightarrow O_t = H_t W_{hq} + b_q$$



Summary of Lesson 1

In Lesson 1, we have:

- ▶ Introduced RNN
- ▶ Discussed models with hidden states
- ▶ Basics of building a RNN

⇒ **References for this lesson:**

Chapter 9, Sections 9.1 to 9.4 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice by building our first recurrent neural network.